1 React `useContext()` – Complete Beginner Notes (Step by Step)

2 Introduction

`useContext` React ko euta Hook ho jasko main purpose **shared (global) data** lai multiple components ma use garna ho, **props manually pass nagari**.

Simple language ma:

useContext lets different components access the same data directly without prop drilling.

---

3 Why do we need useContext?

Suppose timro React application ko structure yesto xa:

```
App
├── Navbar
├── Home
│   └── Product
│       └── ProductDetails
└── Footer
```

Assume `App.jsx` ma user data xa.

```
const user = {
  name: "Manoj",
  age: 22
};
```

`ProductDetails` lai yo user data chainxa.

Without `useContext`, props pass garnu parxa.

```
App
 ↓
Home
 ↓
Product
 ↓
ProductDetails
```

```
<Home user={user} />
<Product user={user} />
<ProductDetails user={user} />
```

Yaha Home ra Product lai user data chaina, tara next component samma pugna props pass garnai parxa.

Yo process lai **Prop Drilling** vaninxa.

---

4 What is Prop Drilling?

Prop Drilling vaneko data chaina bhayeko components le pani data receive garera arko component lai pass gardai janu ho.

Example:

```
App
 ↓
A
 ↓
B
 ↓
C
 ↓
Profile
```

Profile lai matra data chainxa.

Tara A, B, C sabailai props pass garna parxa.

Yo inefficient method ho.

---

5 Solution – useContext

React le Context provide gareko xa.

Context vaneko euta **global storage** ho jasma shared data rakhinxa.

Jun component lai data chainxa tesle direct Context bata data linxa.

Props pass garna pardaina.

---

6 Real Life Example

Imagine office.

```
CEO
 ↓
Manager
 ↓
Team Lead
 ↓
Employee
```

Old method:

CEO → Manager → Team Lead → Employee

Context method:

CEO microphone bata announcement garxa.

Sabai employees directly sunxan.

Manager le manually forward garnu pardaina.

Exactly yesari nai Context le kaam garxa.

---

7 useContext ko 3 Main Steps

React ma Context use garna hamesha yo 3 steps follow garinxa.

```
1. createContext()

↓

2. Provider

↓

3. useContext()
```

---

8 Step 1 – Create Context

Sabai bhanda pahila Context create garinxa.

9 UserContext.jsx

```
import { createContext } from "react";

const UserContext = createContext();

export default UserContext;
```

10 Explanation

`createContext()` le euta empty global box create garxa.

Picture:

```
UserContext

┌──────────────┐
│   Empty      │
└──────────────┘
```

Aile box xa tara data chaina.

---

11 Step 2 – Provider

Provider ko kaam Context vitra value rakhne ho.

12 App.jsx

```
import UserContext from "./UserContext";
import Navbar from "./components/Navbar";

function App() {

  const user = {
    name: "Manoj",
    age: 22,
  };

  return (
    <UserContext.Provider value={user}>
      <Navbar />
    </UserContext.Provider>
  );
}
```

```
export default App;
```

---

13 Explanation

`value={user}`

yo line le user object Context vitra rakhxa.

Picture:

Before

`UserContext`

`Empty`

After

`UserContext`

`name : Manoj`

`age : 22`

Provider = Data Fill Garne.

---

14 Step 3 – useContext()

Aba kunai component le data use garna cha vane `useContext()` use garinxa.

15 Navbar.jsx

```
import { useContext } from "react";
import UserContext from "../UserContext";

function Navbar() {

  const user = useContext(UserContext);

  return (
    <>
      <h1>{user.name}</h1>
      <p>{user.age}</p>
    </>
  );
```

```
}

export default Navbar;
```

---

16 Explanation

```
const user = useContext(UserContext);
```

Yo line le React lai bhanxa:

"UserContext vitra bhayeko value malai deu."

React automatically Provider bata value liyera `user` variable ma store garxa.

---

17 Complete Flow

```
createContext()

↓

Empty Context

↓

Provider

↓

Store Value

↓

useContext()

↓

Read Value
```

---

18 Visual Diagram

App.jsx

```
const user = {
   name:"Manoj",
   age:22
```

```
}
```

```
<UserContext.Provider value={user}>
```

```
UserContext

┌──────────────────┐
│                  │
│ name : Manoj     │
│ age : 22         │
│                  │
└──────────────────┘
```

```
Navbar.jsx

const user = useContext(UserContext);
```

```
user.name

↓

Manoj
```

---

19 What type of data should be stored in Context?

Normally Context ma shared data rakhinxa.

Examples:

- Logged In User
- Authentication
- Theme (Light/Dark)
- Language
- Shopping Cart
- Notification Count
- Currency

- Application Settings

---

20 Advantages of useContext

- Removes Prop Drilling.
- Makes code cleaner.
- Easy to share data across many components.
- Best for global shared data.
- Reduces unnecessary prop passing.

---

21 Limitations

- Frequently changing data ko lagi Context matra best option hudaina.
- Dherai large applications ma Redux, Zustand, wa Context + useReducer use garinxa.
- Provider bahira Context access garna mildaina.

---

22 useContext vs Props

Props	useContext
Parent bata Child samma data pathaunxa	Global shared data access garxa
Manual passing required	Direct access
Prop Drilling huna sakxa	Prop Drilling hudaina
Small communication ko lagi	Shared application data ko lagi

---

23 Important Interview Questions

24 1. What is useContext?

`useContext` is a React Hook that allows components to access shared data directly from a Context without passing props through every intermediate component.

---

25 2. What problem does useContext solve?

It solves **Prop Drilling**.

---

26 3. What are the three steps of Context?

1. `createContext()`
2. `Provider`
3. `useContext()`

---

27 4. What does Provider do?

Provider stores the value inside the Context so that child components can access it.

---

28 5. What does useContext() return?

It returns the value stored inside the nearest matching Provider.

---

29 Summary

- `createContext()` → Creates a global Context.
- `Provider` → Stores data inside the Context.
- `useContext()` → Reads data from the Context.
- Main purpose → Share data across multiple components without prop drilling.
- Common use cases → User Authentication, Theme, Language, Cart, Notifications, Settings.

Golden Formula

```
Need to share data?

↓

createContext()

↓

Provider

↓
```

useContext()

↓

Access shared data anywhere inside the Provider